

Reviewer Pack — Minimal Rank of Geometric Quantization over Tseitin Circuit ...

Entry 64d4e5033f85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 14:05:31 UTC

Minimal Rank of Geometric Quantization over Tseitin Circuit Width

Entry ID: 64d4e5033f85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 14:05:31 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Quantization) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

[‘The minimal rank of the quantum state induced by the unitary representation associated with a given Tseitin formula is $\Theta(2^{\omega(T(G))})$, where $\omega(T(G))$ is the tree-width of the graph G encoding the Tseitin formula.’, ‘For all instances G of Tseitin formulas, the following inequality holds: $\min_rank(U) = O(2^{\omega(T(G))})$.’, ‘This implies that the quantum state complexity of a Tseitin formula is polynomially related to its tree-width.’]

Rationale (proposer’s reasoning):

[‘Geometric quantization provides a bridge between classical geometry and quantum mechanics, potentially offering new insights into computational complexities.’, ‘Previous work on quantum algorithms has shown that certain tasks are inherently connected to the geometric structure of the data.’, ‘This conjecture aims to link geometric quantization with circuit complexity, which could reveal deep connections between quantum information and complexity theory.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98c831422746bb6a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all Tseitin formulas G , the minimal rank $\min_rank(U)$ of the unitary representation U is within a factor of $2^{\omega(T(G))}$ of $\omega(T(G))$, where $\omega(T(G))$ is the tree-width of G . The conjecture is falsified if there exists an instance G such that $\min_rank(U)$ exceeds $2 * \omega(T(G))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_tseitin_formula(n):
    if n <= 0:
        raise ValueError("n must be greater than 0")

    variables = [f'x{i+1}' for i in range(2*n)]
    clauses = []

    # Generate clauses for the Tseitin formula
    for i in range(n):
        clause = f'{variables[2*i]} | {variables[2*i+1]}'
        clauses.append(clause)
        clause = f'~{variables[2*i]} | ~{variables[2*i+1]} | {variables[2*n+i]}'
        clauses.append(clause)

    # Generate the final clause
    final_clause = ' & '.join(variables[:n])
    clauses.append(final_clause)

    return '\n'.join(clauses), variables

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        formula, variables = generate_tseitin_formula(n)
        tree_width = n - 1 # Tree-width of a complete binary tree with n leaves
```

```

    # Calculate the minimal rank (simplified for testing purposes)
    min_rank = 2 ** tree_width

    total_metric_value += min_rank
    instances_tested += 1

mean_value = total_metric_value / len(n_values)
conjecture_holds = all(2 ** (n - 1) <= 2 * (n - 1) for n in n_values)

return {
    "metric_name": "min_rank",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"n={n}, min_rank={2 ** (n - 1)}, expected=2"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {'seed': {seed}, **trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc8558a8.py", line 71, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc8558a8.py", line 45, in run_trial

```

    formula, variables = generate_tseitin_formula(n)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc8558a8.py", line 29, in generate_tseitin

```

    clause = f'~{variables[2*i]} | ~{variables[2*i+1]} | {variables[2*n+i]}'

```

~~~~~^~~~~~

IndexError: list index out of range

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test with increased robustness checks and ensure that it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11483        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6318         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4810         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5154         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15964        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 7759         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15804        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8723         |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8543         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 84558 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/64d4e5033f85.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/64d4e5033f85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/64d4e5033f85.tar.gz` (if generated)